PERSONAL AIR EXPOSURE COMPANION

Three-member execution playbook · dependency-first timeline · coding prompts · GitHub workflow · API-key checkpoints

Purpose: Give the team one ordered plan from repository setup to a tested, demo-ready mobile MVP, so members know what must be finished before the next task starts.

Recommended delivery shape

- **Mobile:** React Native + Expo + TypeScript; Android-first, with manual routine entry as a reliable fallback.
- **Backend:** Python + FastAPI + Pydantic; keep provider integrations and exposure calculations server-side.
- **Data:** one selected air-quality source for the MVP; clearly label measurement/forecast, timestamps, units, and coverage.
- **Storage:** start without a database if the demo can use transient or local routine data; add PostgreSQL only if persistence is genuinely needed.
- **Deployment:** use the cloud/service required by the event rules; verify eligibility before committing to the architecture.

Planning assumption: this playbook uses a 10-working-day schedule. If your actual deadline is shorter, preserve the dependency order and cut optional features—not validation, privacy, or the final submission buffer.

Team roles used throughout this guide

Owner	Primary responsibility	Definition of done
Member 1	Mobile app & location UX	User can enter a routine manually, grant/deny location permission, and view the backend result.
You / Member 2	Backend & exposure engine lead	Stable API contracts, provider integration, validated weighted estimate, coverage and uncertainty handling.
Member 3	Data validation, recommendations & QA	Provider suitability checked, sample data/test cases ready, recommendations grounded, release checklist passed.

Use the prompts as controlled coding tasks. Run them one at a time, inspect the diff, run tests, and commit only verified work. No prompt can guarantee zero bugs; this workflow reduces late integration failures.

1. Non-negotiable execution rules

- **One source of truth:** keep API contracts, environment variable names, and task status in the repository README or /docs.
- **Agree before parallel coding:** create the API contract and sample response before Member 1 builds screens that depend on it.
- **Use mock data before secrets:** UI and calculation tests must work without real API keys. Add live-provider tests only after the basic flow passes.
- **Small changes:** one branch and one focused pull request (PR) per task. Do not push directly to main.
- **Test before merge:** run the relevant checks locally, review the diff, and ask another member to review the PR.
- **Never commit secrets:** API keys, AWS credentials, tokens, .env files, keystores, and signing credentials must stay out of Git history.
- **Location privacy:** request permission only when needed, explain why, provide manual entry, and do not silently track people in the background.
- **Be honest about the science:** this is an estimate based on available outdoor readings/forecasts and reported time. It is not an exact inhaled dose or a guarantee of indoor air quality.

Definition of the MVP

- User can add 2–5 routine stops with approximate duration, manually or with optional foreground location.
- App fetches PM2.5 data for available stops from one provider or returns a clearly marked demo fixture.
- Backend computes a duration-weighted mean over usable segments only.
- UI shows the estimate, units (µg/m³), data timestamp/source, data coverage, and a limitation note.
- A simple recommendation is generated from transparent rules and available evidence—not from an unsupported claim.
- App remains usable when location permission is denied, network fails, or the provider returns missing data.

MVP calculation rule

Weighted mean PM2.5 = Σ(PM2.5■ × duration■) / Σ(duration■ for segments with usable PM2.5 data). Do not replace missing readings with zero. Do not average AQI values as though they were PM2.5 concentrations. If no usable segment exists, return a clear unavailable state instead of a fabricated number.

2. Ten-working-day timeline and dependency flow

The work is deliberately staged: decisions and contracts first, then independent implementation, then integration and release hardening.

Day	Gate / deliverable	Member 1: mobile	You: backend	Member 3: data & QA
1	Scope locked	Review user flow; list screens and permissions	Create architecture, repo plan, API draft	Compare candidate data sources and event constraints
2	Contract agreed	Build UI shell with mock response	Freeze Pydantic models and OpenAPI contract	Prepare sample fixtures and calculation test cases
3	First vertical slice	Routine form + manual entry	Routine validation endpoint	Test input edge cases; verify data units
4	Location path	Foreground permission + coordinates; denial fallback	Location-to-provider adapter interface	Check provider coverage, limits, timestamps, API-key requirements
5	Data working	Call backend using mock/local API	Live provider integration behind adapter	Manually verify live API response and record evidence
6	Exposure engine	Exposure result screen and loading/error states	Weighted estimate + coverage + unavailable state	Independent formula review and tests
7	End-to-end	Connect screens to real backend	Integration fixes; API error handling	Run end-to-end cases; audit labels/limitations
8	Deployment	Test Android device and permissions	Deploy backend; configure secrets/env	Smoke tests against deployed service
9	Release candidate	Polish UX/accessibility and screenshots	Fix bugs; freeze schema; tag candidate	Regression test, privacy review, evidence/demo script

Day	Gate / deliverable	Member 1: mobile	You: backend	Member 3: data & QA
10	Submission freeze	Rehearse app demo; no new features	Verify deployment, logs, env and rollback	Final checklist, README, pitch, submission proof

Dependency gates: Day 2 contract before real frontend integration; Day 4 provider decision before live integration; Day 6 formula tests before trusting results; Day 7 end-to-end pass before deployment; Day 9 release candidate before the Day 10 freeze.

If you have only 5–7 days

- Keep Days 1–2, 3–6, and 7–10 in the same order, but overlap independent tasks after the API contract is frozen.
- Use manual routine entry and one data provider; postpone background tracking, accounts, maps, notifications, historical analytics, and AI chat.
- Prefer a working demo with mocked provider fallback and clearly disclosed limitations over a half-working multi-provider system.

3. Detailed sequence: what must happen before what

DAY 1 — Scope, rules, repository and decisions

Member 1: Sketch 3 screens: Routine, Exposure Result, Settings/Privacy. Specify manual-entry fallback and foreground-only location for MVP.

You: Create repository structure, agree on stack, define API endpoint names and decide whether storage is needed. Keep the backend as the only component holding provider keys.

Member 3: Compare 1–2 air-quality providers: geographic coverage, PM2.5 availability, timestamp, rate limits, terms, key requirement, and whether historic/forecast data is actually available.

Exit gate: Everyone agrees on MVP scope, source choice or source-selection deadline, privacy behavior, event/AWS requirement, and who owns each folder.

DAY 2 — Freeze API contract and build independent mocks

Member 1: Build screen shell with a hard-coded example response matching the agreed contract. Do not wait for live APIs.

You: Create Pydantic request/response schemas, OpenAPI docs, health endpoint, error shape, and a sample response. Commit a contract document.

Member 3: Write known-answer calculation examples, missing-data cases, invalid duration cases, and sample provider JSON fixture.

Exit gate: API schema is version-controlled and no member silently changes field names. If a schema changes, update tests and notify the team in the same PR.

DAY 3 — Routine entry and backend validation

Member 1: Implement add/edit/remove stop, duration input, validation messages, and sample routine state.

You: Implement routine endpoint/model validation: valid coordinates if present, non-negative duration, sensible limits, empty routine rejection, and typed errors.

Member 3: Test boundary values: zero/negative duration, missing location, very long routine, duplicate stops, empty input.

Exit gate: Routine can be entered manually and invalid inputs are rejected consistently by both app and backend.

DAY 4 — Location permission and provider adapter

Member 1: Implement a user-initiated foreground location flow. Explain purpose before asking. If permission is denied or GPS fails, keep manual entry available.

You: Define a provider interface independent of the vendor. Keep location/data lookup on the backend where practical; validate coordinate ranges and normalize provider responses.

Member 3: Verify provider coverage around test locations and document what each response means: observation, forecast, station reading, or interpolated estimate.

Exit gate: Permission-denied path passes. Provider source is selected based on actual coverage and terms, not assumptions.

DAY 5 — Live data and manual API-key verification

Member 1: Use a configurable backend base URL and render loading, empty, success, and failure states.

You: Add live provider adapter and environment configuration. Keep fixture/mock mode for offline development.

Member 3: Perform the manual live API check described in Section 6. Record a redacted sample response, timestamp, units, and result.

Exit gate: Live data request works locally with key stored only in backend environment; mock mode still works without any key.

DAY 6 — Exposure calculation and independent review

Member 1: Render weighted PM2.5 estimate, source/time, coverage and a concise limitations note. Avoid displaying a single number without context.

You: Implement the pure calculation function separately from HTTP/provider code. Add unit tests for weighting, missing readings, no valid segments and invalid values.

Member 3: Independently calculate expected outputs by hand/spreadsheet and compare against automated tests. Review the wording of recommendations.

Exit gate: Known-answer tests pass. No missing reading is treated as zero, and the API returns unavailable rather than inventing a value.

DAY 7 — Integrate the full user journey

Member 1: Connect routine → backend → result. Handle timeouts, permission denial, no network, provider failure, and empty data.

You: Add robust timeout/error mapping, input limits, structured logs without precise personal location unless essential, and API smoke tests.

Member 3: Run the complete checklist on at least one Android device and one test dataset; capture bugs as reproducible issues.

Exit gate: A teammate can complete the main journey without developer intervention. Critical and high-severity bugs are fixed or explicitly block release.

DAY 8 — Deploy and configure safely

Member 1: Point the app to the deployed HTTPS API and test on device using a real network. Verify Android permission prompts.

You: Deploy to the event-approved hosting target; configure environment variables/secrets in hosting settings; verify health endpoint and logs.

Member 3: Run smoke tests against deployment; check CORS if applicable, timeouts, provider quota, privacy wording, and no secrets in client bundle/repo.

Exit gate: Deployed app reaches deployed backend and provider. No production key is bundled into the mobile app.

DAY 9 — Release candidate and rehearsal

Member 1: Polish key flows, accessibility, readable units, loading/error states, and demo reset path.

You: Freeze API contract; review dependency lockfiles, environment template, error handling and deploy instructions. Create a release-candidate tag only after tests.

Member 3: Perform regression checklist, prepare demo script, architecture diagram, data-source explanation, limitations, and screenshots.

Exit gate: No feature work remains. The team rehearses the demo from a clean start and knows the fallback if live data fails.

DAY 10 — Submission freeze

All: Stop feature development. Only fix release-blocking defects. Rebuild/redeploy, run smoke tests, verify README and required submission fields.

You: Confirm deployed URL, environment variable names (not values), backend health, latest commit/tag, and recovery steps.

Member 3: Verify judging/demo assets and submit before the deadline; save confirmation/screenshot where appropriate.

Exit gate: One clean, repeatable demo; source and setup documented; no exposed credentials; all mandatory event requirements checked.

4. Copy-ready coding prompts (run one task at a time)

Paste the relevant prompt into your coding assistant with the repository open. Replace bracketed details only after the team agrees. Each prompt asks the assistant to inspect the existing project first, avoid broad rewrites, and report exactly what changed and how it was tested.

Prompt 0 — Repository-wide rules (use first)

```
You are working in the existing Personal Air Exposure Companion repository. First inspect the current files, package manifests, README, and git status. Do not overwrite or re-scaffold working code without explaining why. Follow the agreed architecture: React Native + Expo + TypeScript mobile app; Python + FastAPI + Pydantic backend; one air-quality provider behind a provider adapter; transparent duration-weighted PM2.5 estimate; manual routine entry fallback; foreground location only for the MVP. Do not add background tracking, authentication, a database, maps, or AI unless a task explicitly requests it. Never hard-code API keys or credentials. Keep secrets in backend environment variables, provide only a .env.example with placeholder values, and ensure .env is gitignored. For each change: list files to change first, implement the smallest coherent patch, add/update tests, run relevant tests/lint/type checks if available, report exact commands and results, list remaining risks, and do not claim tests passed unless you ran them. If requirements conflict or a needed detail is unknown, ask one concise question instead of guessing.
```

Prompt 1 — Architecture, repository skeleton and README

```
Inspect the repository and propose a minimal structure for this app. Create/update README.md and docs/architecture.md with: user flow, member ownership, local setup, environment variables by name only, data flow, privacy behavior, and test commands. Keep mobile and backend folders separate. Add .gitignore entries for .env files, Python virtual environments, node_modules, build outputs, logs, local databases, and credentials. Do not create a fake API key or claim a deployment exists. Add a short checklist of decisions the team must approve before implementation. Run available checks and report results.
```

Prompt 2 — API contract (you own this)

```
Implement the first FastAPI contract only. Inspect existing files first. Create a health endpoint and typed Pydantic request/response models for a routine with stops and durations, plus an exposure report. Include explicit units, data timestamp/source, data type (observation/forecast/estimate/unknown), usable duration, total requested duration, coverage ratio/percentage, and a clear unavailable/error state. Validate coordinate ranges and positive durations; set sensible maximum stop count and input lengths. Keep provider logic and calculation out of route handlers. Add tests for valid and invalid payloads. Generate/update the OpenAPI schema and a docs/api-contract.md with one realistic example. Do not implement live provider calls yet.
```

Prompt 3 — Mobile UI shell with fixtures (Member 1)

```
Inspect the existing Expo app and follow its current routing and styling conventions. Build a minimal mobile-first UI for: (1) entering/editing routine stops and durations, (2) asking for foreground location only after a clear user action with an explanatory message, and (3) displaying an exposure result. Use a local fixture matching docs/api-contract.md; do not call a live air-quality provider from the mobile app. Include loading, empty, validation, error, permission-denied, and no-data states. Manual location/routine entry must work if permission is denied. Do not add background location tracking. Add tests for pure validation/formatting logic where practical and document how to run the app on Android with Expo.
```

Prompt 4 — Routine validation endpoint (you own this)

```
Using the frozen API contract, implement routine input validation in FastAPI. Keep the validation logic testable and independent of the provider. Reject empty routines, non-positive or unreasonable durations, invalid latitude/longitude, unsupported fields where appropriate, and excessive stop counts. Return consistent structured validation errors. Add tests for normal input, missing values, zero/negative durations, out-of-range coordinates, and upper limits. Do not store exact location history or add a database unless the current requirements explicitly need it.
```

Prompt 5 — Provider research and adapter (Member 3 + you)

```
Before coding, inspect docs/ and the selected provider's official documentation supplied by the team. Summarize whether the provider supports our target geography, PM2.5 concentration, timestamp, observation vs forecast distinction, rate limits, API key requirement, and terms. Do not assume an endpoint or field name. Then implement a provider adapter with a typed normalized result and a fixture/mock implementation. Keep vendor-specific JSON parsing in one module. Add tests using a redacted fixture and cases for missing PM2.5, malformed JSON, HTTP error, timeout, stale timestamp, and unit normalization. If documentation is unavailable, stop and list the exact facts that require manual verification.
```

Prompt 6 — Live provider integration (you own this)

```
Implement live calls only after the team has verified the selected provider's official endpoint and key requirements. Read the API key from a backend environment variable named in .env.example; never put it in source code, app config, logs, screenshots, or client requests. Use a timeout, bounded retry policy for transient failures, response validation, rate-limit/error handling, and safe logging that does not print secrets or unnecessary precise coordinates. Keep mock mode available when no key is configured. Add tests with mocked HTTP responses; do not make automated tests depend on the real network. Clearly label returned data as observation, forecast, or estimate according to the provider response—never guess. Update README setup steps.
```

Prompt 7 — Exposure engine (you own this; independent review by Member 3)

```
Implement a pure, deterministic function that computes duration-weighted mean PM2.5 concentration from routine segments with valid PM2.5 concentration and positive duration. Formula: sum(concentration × duration) / sum(duration for usable segments). Do not treat missing data as zero, do not mix AQI with PM2.5 concentration, and do not label the result as exact inhaled dose. Return usable duration, requested duration, coverage ratio, excluded-segment reasons, units, and an unavailable state if no valid segment exists. Validate negative/non-finite concentrations and invalid durations. Add unit tests including 2 hours at 100 plus 8 hours at 25 = 40 µg/m³; one missing segment; all missing; zero duration; malformed values; and rounding/display precision. Ask for a second-person review of expected values.
```

Prompt 8 — Mobile-to-backend integration (Member 1 + you)

```
Connect the existing mobile UI to the backend using the frozen API contract. Use a configurable API base URL that differs between local emulator/device and deployed HTTPS backend; do not assume localhost works from a physical phone. Do not call the air-quality provider directly from the app. Implement request timeout, loading state, retry action, validation messages, no-data state, and readable network/server errors. Preserve manual entry if location permission is denied. Add tests for response parsing and formatting. Document how to set the API base URL without committing secrets. Run type checks/tests and list any manual device tests still required.
```

Prompt 9 — Recommendations and safety wording (Member 3)

```
Review the current exposure result fields and implement a small set of transparent, rule-based suggestions grounded in the available data. Do not invent medical claims, diagnose a person, promise health protection, or infer indoor air quality from outdoor readings. If coverage is low, data is stale, or the provider result is unavailable, prioritize explaining the limitation rather than giving a confident recommendation. Keep language practical and non-alarmist. Add tests for low-coverage, stale-data, unavailable-data, and normal cases. Include a visible note that the estimate is based on available readings/forecasts and reported durations, not exact inhaled dose.
```

Prompt 10 — End-to-end tests and bug hunt (all; Member 3 coordinates)

```
Inspect the whole repository and the API contract. Do not refactor unrelated code. Create a test plan and execute available unit tests, backend tests, lint/type checks, and build checks. Add or fix tests for the complete path: manual routine entry → API validation → provider fixture/live adapter → weighted estimate → UI result. Cover permission denied, no network, timeout, provider rate limit, stale/missing data, invalid routine, no usable PM2.5 segments, and successful result. For each failure, verify the user sees a clear state and no fabricated number. Report exact commands, pass/fail outcomes, untested items, and reproducible bug steps. Never report a check as passing unless executed.
```

Prompt 11 — Deployment readiness and secrets audit (you own this)

```
Audit the repository for deployment readiness without printing secret values. Check for accidental API keys, credentials, .env files, client-side provider keys, debug-only URLs, permissive CORS, and logs containing sensitive location data. Use the hosting provider's documented environment-variable mechanism. Verify health endpoint, HTTPS URL, startup command, dependency versions/lockfiles, timeout behavior, and a smoke-test command. Do not deploy or delete resources without asking first. Update README with deployment steps and variable names only. Produce a release checklist and list any event-specific AWS/open-source requirement that still needs manual confirmation.
```

Prompt 12 — Final code review (another member reviews each PR)

```
Review this pull request against the agreed MVP scope and API contract. Look for correctness bugs, missing-data handling, unhandled errors, security/privacy issues, accidental secrets, accessibility/usability gaps, inconsistent units, weak tests, unnecessary dependencies, and changes outside the task. Do not rewrite the PR. Return findings ordered by severity with file/line references and concrete fixes. If no blocker is found, say what you verified and what still requires manual testing. Do not assume a passing build proves location permissions or live provider behavior work on a real device.
```

5. GitHub workflow: what to push, when, and how

Recommended repository layout

```
personal-air-exposure-companion/
mobile/
backend/
docs/
architecture.md
api-contract.md
data-source-notes.md
test-plan.md
demo-script.md
.gitignore
README.md
.env.example
```

Branch and PR rules

- Protected shared branch: **main**. Keep it runnable and demoable.
- Branch names: **feat/mobile-routine-form**, **feat/api-contract**, **feat/provider-adapter**, **test/exposure-engine**, **fix/no-data-state**, **docs/demo-guide**.
- One task per branch; commit focused changes with descriptive messages such as **feat: add routine validation endpoint**.
- Before opening a PR: sync with main, run tests, inspect **git diff**, check status for .env/secrets, and describe manual test steps.
- Require at least one teammate review. Merge only when checks pass and API/schema changes have been communicated.
- After merge, pull main and run the app before starting the next dependent task.

Milestone push checklist

When	What must be pushed	Required review / gate
End Day 1	README skeleton, .gitignore, architecture draft, folder structure	All members approve stack, scope and owners
End Day 2	API schemas, docs/api-contract.md, mock response, sample fixtures	Mobile and backend fields match exactly
End Day 3	Routine UI, validation endpoint and tests	Invalid input cases pass
End Day 4	Foreground permission flow, provider interface, source notes	Manual fallback works; source facts documented
End Day 5	Provider adapter, mock mode, redacted fixture and tests	Manual live test recorded; no key committed
End Day 6	Exposure engine and known-answer tests	Independent expected-value review passes
End Day 7	End-to-end integration and test plan	Main user journey passes
End Day 8	Deployment instructions, health endpoint, smoke-test evidence	HTTPS deployed app reaches backend
End Day 9	Release candidate tag, demo script, screenshots, final README	Regression suite and secrets audit pass
Day 10	Final source, submission assets, deployment URL if required	Freeze: only critical fixes

Useful Git commands

```
git status
git switch -c feat/short-task-name
git add path/to/changed/files
git diff --cached
git commit -m "feat: describe the change"
git push -u origin feat/short-task-name
# Open a PR; request one teammate review; merge only after checks pass.
git switch main
git pull --ff-only
```

Secret recovery rule: if a key is ever committed, deleting it in a later commit is not enough. Revoke/rotate it immediately, remove it from history using the team's approved procedure, and audit logs. Never paste a secret into an AI prompt, issue, PR, or chat.

6. Manual verification and API-key checkpoints

Do not request or paste real keys until the team has selected a provider and read its official documentation. Exact endpoints, authentication headers, quotas and access rules change; verify these from the provider's current docs rather than copying guessed examples.

Checkpoint	What to verify manually	Who / where the key belongs	Pass condition
Provider selection (Day 1–4)	PM2.5 concentration exists for target city; source timestamps/units and observation-vs-forecast meaning; coverage; rate limits; terms; key requirement	Member 3 documents facts; you approve adapter contract	Source is suitable for the demo and its limitations are known
Live API test (Day 5)	Create/obtain key only if required; call documented endpoint for a known test location; inspect HTTP status, JSON fields, units, timestamp and missing-data behavior	Backend .env/local environment or secret manager only	One live request parses correctly; key is never logged or committed
Local backend (Day 5–7)	Set variable named in .env.example; restart server; test live mode and mock mode without key	Your backend terminal/IDE environment	Live mode works with key; mock mode works without key
Mobile app (Day 7)	Confirm app calls backend URL, not provider; inspect bundle/source/config for provider key	No provider key in mobile app; only non-secret API base URL	App works without exposing provider credential
Deployment (Day 8)	Add provider key to hosting environment/secrets settings; redeploy/restart; inspect redacted logs and health endpoint	Deployment secret manager/dashboard	Deployed backend fetches data and logs contain no key
AWS/event requirement (Day 1 and Day 8)	Read current event rubric: qualifying AWS service, open-source requirement, proof/screenshots, deployment restrictions and account setup	Team lead verifies; AWS credentials handled privately	The solution satisfies actual rules; no assumption based on this guide

Environment-file example (placeholders only)

```
# backend/.env.example — safe to commit; values are placeholders
AIR_QUALITY_API_KEY=replace_with_your_own_key_if_provider_requires_one
AIR_QUALITY_BASE_URL=verified_provider_base_url
DATA_MODE=mock
APP_ENV=development
# Keep real values in backend/.env (gitignored) or the deployment secret manager.
# Do not add provider keys to mobile/.env, Expo public config, source code, README, or screenshots.
```

Manual verification checklist for a live provider call

- Read the current official docs; confirm the correct endpoint, authentication method, response schema, rate limit, and usage terms.
- Use a test location that the provider actually covers; verify units are concentration units, not AQI.
- Record source name, response timestamp, data type, PM2.5 value (if present), and a redacted sample response.
- Test at least one missing/stale/error response. The app must show unavailable/limited data, not zero.
- Verify credentials do not appear in terminal logs, app bundle, screenshots, Git diff, issue tracker, or PR.
- If a key is optional/not required, don't invent one; if access approval is pending, keep fixture mode and mark live verification as pending.

7. Quality gates: tests that must pass before submission

Area	Required test	Pass condition
Math	2h × 100 + 8h × 25	Weighted mean is 40 µg/m³
Missing data	One segment has no usable PM2.5	Exclude it from numerator and denominator; report coverage
No data	All segments missing or provider unavailable	No numeric exposure estimate is fabricated
Validation	Negative duration, out-of-range coordinates, empty routine	Consistent validation error; app explains what to fix
Location	User denies permission; GPS unavailable	Manual routine entry still works
Network	Backend timeout / offline device	Clear retryable error; no crash or false value
Provider	Rate limit, malformed response, stale timestamp	Safe error/limited-data state; secret not logged
Privacy	Review permissions and logs	No silent background tracking; minimize location logging
Security	Search repo/history diff for keys	No active credentials committed; any leak rotated
Mobile	Install/run on real Android device	Routine, permission, API call and result all work
Deployment	Smoke test deployed HTTPS endpoint	Health check and main API work from device
Demo	Run demo twice from clean start	No developer-only manual edits needed mid-demo

Minimal commands (adapt to the actual files)

```
# Backend: from backend/ after activating the virtual environment
python -m pip install -r requirements.txt
python -m pytest
uvicorn app.main:app --reload

# Mobile: from mobile/
npm install
npx expo start
# Run the lint/type-check/build scripts actually defined in package.json.
# Do not assume these commands exist; inspect package.json first.
```

Manual device testing is mandatory

Automated tests cannot fully verify real Android permission prompts, GPS behavior, network routing from a physical phone, or the exact live-provider coverage. Test these manually on a real device and record the result.

8. Data contract and product wording

Every exposure result should carry context

- **Estimate:** weighted PM2.5 concentration in µg/m³ over usable segments, with rounding documented.
- **Coverage:** usable duration ÷ requested duration. If segments have different confidence, don't imply the simple duration ratio is a complete measure of uncertainty.
- **Provenance:** provider/source name, source timestamp, retrieval timestamp if available, and whether the value is an observation, forecast, or estimate.
- **Limitations:** outdoor station/forecast values may not represent the exact route, indoor environment, personal breathing rate, or inhaled dose.
- **Recommendations:** use transparent rules and avoid medical diagnosis or guaranteed outcomes.
- **Location:** ask for permission in context; provide manual input; avoid collecting/storing location history unless the MVP requires it and users understand it.

Suggested API response shape (illustrative; finalize in docs/api-contract.md)

```
{
"status": "ok",
"estimate": {
"metric": "PM2.5",
"value": 40.0,
"unit": "µg/m³",
"method": "duration_weighted_mean"
},
"coverage": {
"requested_duration_minutes": 600,
"usable_duration_minutes": 600,
"ratio": 1.0
},
"source": {
"name": "selected provider",
"data_type": "observation_or_forecast",
"timestamp": "provider timestamp"
},
"limitations": [
"This is an estimate based on available data and reported durations, not exact inhaled dose."
]
}
```

The real response must be validated against the actual provider and agreed Pydantic models. The example is a shape illustration, not a claim that a provider returns these fields.

9. Submission package and final 24-hour freeze

README must let a new teammate run the project

- Project summary, features, architecture diagram, and explicit MVP limitations.
- Prerequisites and exact setup commands for backend and mobile.
- Environment variable names and where to configure them—never actual values.
- How to run with mock data and how to enable live data after manual key setup.
- Test commands and latest known results; distinguish automated from manual tests.
- Data-source attribution, source timestamp meaning, coverage limitations and usage terms.
- Privacy/permissions explanation, screenshots, demo script, deployed URL if required.
- Known issues and what is intentionally out of scope.

Final checklist — tick every item

■ Scope and event eligibility reviewed; qualifying AWS/open-source requirements verified from current official event rules.

■ main branch is clean, latest changes are pushed, PRs reviewed and merged.

■ No .env, API keys, AWS credentials, access tokens, or signing secrets in repo or mobile bundle.

■ Backend unit tests and validation tests pass.

■ Exposure formula independently checked; missing data never becomes zero.

■ Mobile app runs on a real Android device; permission-denied/manual path works.

■ Live provider test completed or clearly marked as not verified; mock fallback works.

■ Deployed backend responds over HTTPS; app points to the correct base URL.

■ Smoke test, error states, no-data state, and retry flow verified.

■ README, architecture, API contract, data-source notes, screenshots and demo script are current.

■ Demo rehearsed twice from clean start; fallback demo data is ready and clearly labelled.

■ Submission confirmation saved before the deadline; no unreviewed feature changes after freeze.

Release rule: after the release candidate is approved, add no new features. Only fix a confirmed release-blocking bug, rerun the affected tests, and document the change.

10. Official documentation to check

Verify current details in official documentation before installing packages, configuring provider access, or deploying. This guide intentionally avoids hard-coding a specific provider endpoint or event rule.

Expo documentation: https://docs.expo.dev/

Expo Location: https://docs.expo.dev/versions/latest/sdk/location/

Expo Go: https://expo.dev/go

Android Studio: https://developer.android.com/studio

React Native: https://reactnative.dev/docs/environment-setup

Node.js downloads: https://nodejs.org/en/download

Python downloads: https://www.python.org/downloads/

FastAPI: https://fastapi.tiangolo.com/

Pydantic: https://docs.pydantic.dev/latest/

OpenAQ API documentation: https://docs.openaq.org/

CPCB official portal: https://cpcb.nic.in/

AWS Free Tier: https://aws.amazon.com/free/

AWS Pricing Calculator: https://calculator.aws/

GitHub Docs: https://docs.github.com/

A note on source choice

OpenAQ and national/regional sources are candidates to investigate, not a guarantee that a particular city, coordinate, forecast, endpoint, or key tier will work. Confirm actual coverage and terms for the locations used in the demo. Do not design the app around a provider field until its current official documentation has been checked.

End of playbook · Build the smallest trustworthy version first.